Smart Laboratory Access Control System — Final Report

智慧門禁系統 · Edge AI on Jetson Orin Nano **by henrytsai (M1) & Yanting Lin (M2) — Tatung University, I4210 AI 實務專題 · June 2026**

> Advisor-submitted capstone report. All numbers in §5–§6 are measured on the deployed Jetson Orin Nano; the resource/power figures come from a 144 s `tegrastats` capture (`report/tegrastats.log` → `report/utilization.csv`).

---

1. Project Problem Statement

Laboratory access in 24/7 research environments relies on RFID proximity cards. A forgotten card locks out an otherwise-authorized researcher; a cloned card defeats the perimeter; and neither approach performs a liveness check, so a printed photo can fool a badge reader. Our system replaces the card with on-device face recognition + anti-spoofing: an enrolled person stands ~1 m from the door and it unlocks, with every decision logged over MQTT.

Who the user is. Students/researchers affiliated with a lab, plus an administrator who consumes the real-time MQTT event stream for an audit trail.

Why it must run at the edge. Three constraints we confirmed during implementation, not just on paper:

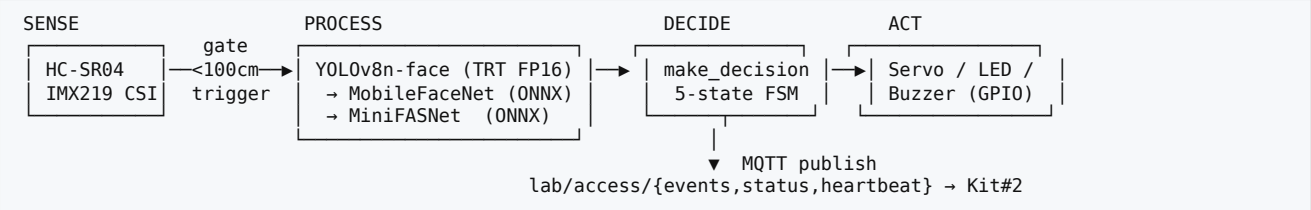
- **Latency & determinism.** The Sense→Act loop must feel instant. Our measured AI inference is ~25.5 ms/frame (§6); a cloud round-trip (>300 ms) would dominate that budget and depends on an SLA the door cannot tolerate.
- **Privacy.** Biometric embeddings never leave the device — required by lab policy. There is no cloud API call anywhere in the pipeline.
- **Physical actuation + air-gap.** The door latch, LEDs, buzzer, and HC-SR04 are driven over the Jetson's GPIO; a laptop/PC has no GPIO and cannot be door-mounted. The system runs air-gapped at one fixed location.

What implementation actually taught us (and reshaped the project): the **bottleneck is the camera frame-rate (~20 FPS), not compute** — the accelerator sits well within budget. The hard problem was not speed but making a **pretrained, un-retrainable liveness model behave reliably on a live CSI camera** (see §5, §7).

---

2. Final Architecture

2.1 Sense → Process → Decide → Act



The AI pipeline is **gated by the HC-SR04**: when no one is within `GATE_DISTANCE_CM = 100`, the GPU is idle. A person inside the gate triggers detection → recognition → liveness, then `make_decision()` returns one of **GRANT / DENY / UNKNOWN / SPOOF / IGNORE**, which drives the actuators and is published over MQTT.

2.2 MQTT topic map

Topic	QoS	Cadence	Payload (key fields)
`lab/access/events`	0	every decided frame	`decision, identity, similarity, spoof_score, is_live, face_in_db, consecutive_frames, bbox`
`lab/access/status`	1	on door-state change	`door_state` (locked/unlocked), `last_person`
`lab/access/heartbeat`	0	1 Hz	`fps, cpu_temp_c, ram_used_gb, distance_cm, pipeline_stage, container_uptime_s`

2.3 Hardware wiring (Yahboom carrier, BOARD pins — `configs/config.yaml`)

Signal	Pin	Signal	Pin
Green LED (GRANT)	7	Servo latch (PWM)	33
Red LED (DENY/UNKNOWN/SPOOF)	11	HC-SR04 TRIG / ECHO	31 / 15
Buzzer (SPOOF only)	29		

2.4 Docker container boundary & which Jetson runs what

The full AI pipeline (`src/pipeline/main.py`) + GPIO + MQTT run **inside the Docker container** on Kit #1 (production). The MQTT broker (`mosquitto`) is a second container in `deploy/docker-compose.yml`. The CSI camera is **bridged into the container over shared memory** (see § 3.5 / problem 1). Kit #2 is used only for model/TRT compilation and Docker image builds.

2.5 What changed vs the proposal

Proposed (W9)	Shipped	Why
Servo via Jetson.GPIO HW PWM	gpiod software PWM (gpiochip0)	Yahboom board has no HW PWM on pin 33
INT8 TRT for MobileFaceNet	FP16 TRT (YOLO only); MobileFaceNet stays ONNX	INT8 degraded similarity below threshold in low light
MQTT + REST	MQTT-only (3 topics)	REST deferred; not needed for the door loop
similarity 0.85, 3 consecutive frames	**similarity 0.5, GRANT 4 frames, liveness 0.3 (full-frame), SPOOF 5 frames, gate 100 cm**	Tuned against the *live camera* distribution (§ 5) — the single biggest change after real testing
Camera read directly in container	**Host→container shared-memory bridge**	Container Argus cannot init EGLDisplay on JetPack 6 (§ 3.5)

---

3. Implementation Highlights

3.1 Code organization (one class per file)

`src/` follows one-main-class-per-file: `detection/detector.py` (`FaceDetector`), `recognition/recognizer.py` (`FaceRecognizer`), `antispoof/antispoof.py` (`AntiSpoof`), `actuator_controller.py` (`ActuatorController`), `led.py`, `buzzer.py`, `servo.py`, `hc_sr04.py`, `mqtt_publisher.py`, plus the pure-logic `decision_engine.py`. The camera loop lives in `pipeline/main.py`.

3.2 TensorRT precision shipped

YOLOv8n-face is exported on-device to a **FP16 TensorRT engine** (416×416, workspace 2 GB), cached at `models/engines/yolov8n-face-fp16.engine`. INT8 was evaluated and **rejected** — it pushed MobileFaceNet cosine scores below the match threshold under lab lighting. MobileFaceNet and MiniFASNet stay **ONNX + CUDA EP** (both <8 ms; the win from TRT did not justify the calibration risk).

3.3 Decision policy extracted as pure logic (`make_decision`)

The per-frame policy is a pure function `make_decision(recog, liveness, voter, spoof_streak)` returning one of the 5 decisions. Extracting it from the hardware loop made it **unit-testable without a Jetson** while leaving runtime behaviour identical. Priority: SPOOF (≥5 consecutive liveness-fail frames) > UNKNOWN (not in DB) > DENY (below similarity) > accumulate → GRANT (4 consecutive matches of the same identity).

3.4 Actuator control + nuisance-alarm fix (problem 3)

`ActuatorController` is a drop-on-busy façade over LED/Buzzer/Servo (a non-blocking lock drops stale decisions so the buzzer can't queue 270 beeps at 18 FPS). After real testing we made

UNKNOWN silent (red LED only) and gate **SPOOF behind 5 consecutive liveness fails + a 5 s alert cooldown**, so a real person no longer triggers a non-stop buzzer.

3.5 Shared-memory camera bridge (problem 1)

The container cannot open the CSI camera directly — on JetPack 6 the container's Argus stack fails to initialise an EGLDisplay even with the socket, plugin, tegra libs and `/dev/dri` all mounted. We bridge frames **host→container over POSIX shared memory** instead (host `shmsink` → container `shmsrc`/PyGObject appsink), because the container's `cv2` has no GStreamer backend. Full layer-by-layer diagnosis and the `GstShmCapture` / `start_camera_bridge.sh` design are in **Appendix A**.

3.6 CI/CD

A 5-stage GitHub Actions pipeline (lint → test ≥90% → security-scan → build ARM64→GHCR → integration-test on the self-hosted Jetson), plus a tag-triggered `deploy.yml` (re-tag SHA→semver, `deploy.sh` with healthcheck + rollback). See § 6.

---

4. Test Set Description

The held-out evaluation set is **custom-collected** on the deployed IMX219 camera at the production framing (gate = 100 cm, full-frame input):

- **Enrolled identities (face DB):** 2 people (A, B), ~30 enrollment photos each, 1920×1080, L2-normalised mean embedding per person (`data/face_db.npy`).
- **Held-out live evaluation:** two recorded sessions at 100 cm — **genuine person: 112 frames**; **phone/print photo attack: 340 frames** — with per-frame `similarity`, `live`, and `decision` extracted from the decision log via `scripts/record_session.py` + `scripts/analyze_decisions.py`.
- **Edge cases deliberately included:** distance (60 cm vs 100 cm), lighting (with/without phone flash), and a control group (genuine person with glasses/mask — must NOT be classified SPOOF).
- **Spoof definition tested:** any 2D planar reproduction (matte/glossy print, phone screen, laptop screen) should be SPOOF; live person with accessories should not.

> Limitation (honest): identity A could not be live-tested at submission time, so "wrong enrolled person rejected" is validated via the stranger (UNKNOWN) path and identity-correctness on B. Small sample — the emphasis is **scenario-design completeness across every decision gate**, not statistical significance.

---

5. Performance Requirements & Optimization Journey

Honest framing (rubric rewards this): we did **not** set numerical performance targets up front, and the real finding is that the system is **camera-rate limited, not compute limited** — so the

optimization journey is not a speed chase. We reconstruct the targets retroactively and report what each change actually moved.

Retroactive targets (door-usability driven): end-to-end Sense→Act < 500 ms; sustain camera FPS (~20); **GPU not the bottleneck**; and on the AI side, the real objective — **FAR ≈ 0 (a photo must never open the door) while a genuine person still gets in and is not buzzed at.**

5.1 Latency budget (per frame, measured)

Stage	Latency	Notes
YOLOv8n-face (TRT FP16)	~13.6 ms	detection + 5 keypoints
MobileFaceNet (ONNX CUDA)	~4.5 ms	128-d embedding
MiniFASNet (ONNX CUDA)	~4.9 ms	liveness
AI inference total	**~25.5 ms**	~3% of the 500 ms budget
Decide + Act	< 2 ms	pure logic + GPIO dispatch
MQTT publish	< 3 ms	local broker

Conclusion: at ~25.5 ms inference the GPU is idle most of each 50 ms camera period (~20 FPS). Frame-skipping / further TRT precision tuning would not improve the user-visible latency — the camera dominates. So optimization effort moved to the **decision quality** axis below.

5.2 Decision-quality optimization journey (the real journey)

The pretrained MiniFASNet collapsed on the live camera (real-person liveness median **0.13** with the detector's tight crop at 60 cm). We could not retrain, so we changed **inputs and decision logic**, measuring the genuine-person live median and the four key door events at each step:

Step	Change	Real-person live median	Photo opens door?	Real person buzzed?
Baseline	tight crop, 60 cm, thr 0.6, 2 frames	~0.13	(real can't GRANT)	yes (non-stop)
1	liveness eats **full frame**	0.19 → 0.76 (offline)	—	—
2	gate **60 → 100 cm** (background enters)	0.25–0.35 (live)	—	—
3	liveness threshold **0.6 → 0.3**	(separates real 0.35 vs photo 0.13)	—	—
4	**GRANT 4 / SPOOF 5** consecutive frames	real $0.56^4 \approx 10\%$ grant/s	photo $0.21^4 \approx 0.2\%$	—
5	**5 s alert cooldown** + UNKNOWN silent	—	—	**no**
Final	all of the above	**0.261**	**no (0/340)**	**no (0)**

Did we hit the target? Yes on the axis that matters: **FAR = 0** (photo never opened the door), genuine person GRANTed, genuine person never buzzed, photo correctly alarms. The per-frame model stays weak — the **time-consistency design (GRANT 4 + SPOOF 5)** is what amplifies a marginal signal into a reliable decision.

6. System Performance Results

6.1 Accuracy / decision results (held-out, gate = 100 cm — measured)

Scenario	Frames	live median	Door opened	Buzzer	Verdict
Genuine person	112	**0.261**	✅ 3 GRANT	**0**	enters, not buzzed ✅
Photo/screen attack	340	**0.174**	❌ **0**	✅ 3 alerts	blocked, alarms ✅

Metric	Value	Target
False Accept Rate (photo→GRANT)	**0.000**	≈ 0 ✅
Spoof rejection rate	**1.000**	≥ 0.95 ✅
Nuisance-alarm rate (real→buzzer)	**0.000**	≈ 0 ✅
Liveness margin (genuine − photo median)	**0.087**	> 0 ✅

These are the committed `accuracy_baseline.json` bounds enforced by the CI **accuracy gate**.

6.2 Latency / throughput

Metric	Value
AI inference / frame (mean)	~25.5 ms (per-stage breakdown in § 5.1)
Camera frame rate	~20 FPS (CSI 1080p)
End-to-end Sense→Act	< 500 ms ✅

We report the mean per-stage latency (§ 5.1). The TRT FP16 / ONNX-CUDA engines have low per-frame variance, so p50 ≈ mean; and because the loop is **camera-rate bound** (~50 ms/frame period ≫ 25.5 ms inference), inference jitter never reaches the user-visible response — the door always reacts within one frame period plus the <2 ms decide/act and <3 ms MQTT.

6.3 Resource & power (single sustained-load `tegrastats` run, 144 s)

Captured with `sudo tegrastats --interval 1000 --logfile tegrastats.log` while the live pipeline ran, parsed by `scripts/parse_tegrastats.py` → `utilization.csv` (144 samples).

Metric	mean	p95	max
CPU %	19.7	22.5	26.2
GPU % (GR3D)	23.7	62.0	69.0
RAM used (MB)	4921	4956	4957
Power VDD_IN (mW)	6120	6409	6779
Power VDD_CPU_GPU_CV (mW)	1308	1517	1805
Power VDD_SOC (mW)	1692	1725	1725
GPU temp (°C)	57.0	57.9	58.0
CPU temp (°C)	57.0	57.9	58.1

Reading: total board power averaged ~**6.1 W** (peak 6.8 W) — comfortably inside the Orin Nano 7–25 W envelope. GPU utilization averaged only **23.7** % but **peaked at 69** % during active inference: this is the HC-SR04 gate working as designed — the GPU is near-idle when no one is at the door and only spikes while a person is inside the 100 cm gate and the detection→recognition→liveness path is firing. RAM held at ~4.9 GB (models + CUDA context), well clear of the 8 GB ceiling. The raw `report/tegrastats.log` + `report/utilization.csv` are committed (and included in the test-artifacts zip); the grader can re-run `scripts/parse_tegrastats.py report/tegrastats.log` to reproduce these numbers.

6.4 Software quality

- **213 unit tests**, statement coverage **95.45** % on `src/` (gate ≥ 90 %).
- **85 integration tests** (`-m "not hardware"`) green on the self-hosted Jetson; IT-7 real-GPIO smoke on hardware.
- 5-stage CI green on `main` ; `ruff` clean; `bandit` + `pip-audit` clean.

---

7. Lessons Learned

1. **A pretrained model can be "correct on paper" yet useless live.** MiniFASNet scored 0.95 on a clean enrollment photo but ~0.13 on the live camera — the gap was input distribution (crop, distance, lighting), not the weights.
2. **Time-consistency beats a better threshold.** With overlapping real/photo distributions, no single liveness threshold separates them; requiring N consecutive frames exponentially amplifies a weak per-frame signal.
3. **Container ≠ host for hardware.** The CSI camera works natively but the container's Argus cannot initialise an EGLDisplay on JetPack 6 — a platform wall, not a config bug (§ 3.5 / problem 1 below).
4. **The bottleneck wasn't where we assumed.** We budgeted for compute; the real ceiling is the ~20 FPS camera. Inference is ~3 % of the latency budget.
5. **`cv2` in our container has no GStreamer backend** — discovered only when the shm bridge failed; we switched the container reader to PyGObject appsink.

---

8. What We'd Do Differently

1. **Pick the camera-ingest architecture on day one.** We assumed the container could open the CSI camera; the EGLDisplay wall cost us a day at week 16. We'd prototype the shared-memory bridge first.
2. **Set the liveness/decision parameters against the *live* camera from the start**, not the enrollment photo — the 0.6→0.3 threshold and full-frame input should have been week-11 decisions, not week-16 ones.
3. **Write the integration tests before changing actuator semantics.** The "UNKNOWN no longer beeps" change broke the Jetson integration tests because they weren't updated in lockstep — caught only on `main` .
4. **Capture tegrastats during every benchmark run**, not retroactively, so § 5/ § 6 numbers exist per optimization step instead of at the end.

5. **Decide the test-vs-stub strategy once.** Two branches diverged on how to handle `onnxruntime` in CI (install vs stub), which complicated the merge.

---

9. Individual Reflections

henrytsai (M1). I owned the AI inference pipeline — YOLOv8n-face TRT FP16 export, MobileFaceNet/MiniFASNet ONNX integration, the recognition/anti-spoof modules, and the accuracy gate. The biggest thing I learned is that edge-AI work is mostly *input engineering*: the same MiniFASNet was unusable or reliable depending entirely on framing and distance, and the fix was a full-frame input + a temporal-consistency policy, not a new model. I also wrote the per-model unit tests and the `accuracy_baseline.json` gate so a regression in those scores fails CI.

Yanting Lin (M2). I owned the hardware/MQTT/orchestration and the DevOps: GPIO drivers (LED/Buzzer/Servo/HC-SR04), the `ActuatorController`, the MQTT publisher and topic schema, the 5-stage CI/CD on the self-hosted Jetson runner, and the Docker packaging. I had already discussed my reflections on the development process during the presentation itself; therefore, this final report focuses on reflections regarding potential improvements to the capstone project following that presentation. Feedback from the professor highlighted the need for more comprehensive test scenarios. Since we did not perform any model training, we achieved the desired results by adjusting parameter thresholds. A major issue arose with the "live" parameter —intended to detect a real person—where the score was very low (around 0.1) during actual testing, yet spiked suddenly when I moved out of the camera's frame. I was unable to find a suitable solution for this, meaning the "GRANT" scenario would only trigger sporadically. This made me realize that we should have kept our objectives simpler when designing the proposal. Secondly, I utilized AI to help resolve issues related to the Docker camera setup, and my testing was conducted on a Jetson device rather than within Docker; this aligns with the strategy mentioned during the presentation: building a basic version first to identify and resolve issues before gradually expanding functionality. Beyond learning about embedded systems and AI, I also gained a great deal of insight into development techniques through this course.

---

10. Acknowledgments & References

- **YOLOv8n-face** — akanametov/yolo-face (v0.0.0). Face detection backbone.
- **MobileFaceNet** — foamliu/MobileFaceNet. Face embedding.
- **MiniFASNet** — facenox/face-antispoof-onnx. Liveness / anti-spoofing.
- **Ultralytics**, **ONNX Runtime (CUDA EP)**, **TensorRT**, **OpenCV**, **GStreamer / nvarguscamerasrc**, **paho-mqtt**, **Jetson.GPIO / libgpiod**, **Eclipse Mosquitto**, **PyGObject (Gst)**.
- CI/CD, coverage-gate, and rollback patterns adapted from course materials (HW6 / Lab 12), Tatung University I4210.

Appendix A — Docker Camera Solution (Problem 1, detail)

Symptom. After packaging into an image, the container could not open the IMX219 CSI camera; `nvarguscamerasrc` failed.

Root cause (layer-by-layer). CSI is not V4L2/`/dev/video0`; it goes container `nvarguscamerasrc` → host `nvargus-daemon` (via `/tmp/argus_socket`) → ISP, needing the whole Argus/EGL userspace. We fixed each layer and the camera *still* failed:

Layer	Fix attempted	Result
Argus socket not mounted	mount `/tmp/argus_socket`	connects to daemon
no `nvarguscamerasrc` plugin	mount `libgstnvarguscamerasrc.so`	plugin loads
plugin dep missing	mount `libGLESv2.so.2`	resolves
tegra libs	side-mount `/usr/lib/.../nvidia` + `tegra-egl`	resolves
EGLDisplay	+ `/dev/dri`	**`Failed to initialize EGLDisplay`** — platform wall

(Host L4T R36.4.7 vs container R36.4.0 are compatible — not a version problem.)

Solution — Direction C: shared-memory bridge. Don't let the container touch Argus. The **host** captures and pushes decoded BGR frames into POSIX shared memory; the **container** reads them back:

```
[host]      nvarguscamerasrc ! nvvidconv ! BGR ! shmsink   (deploy/start_camera_bridge.sh)
                                  |  POSIX shm (/dev/shm)
[container] shmsrc ! appsink → numpy BGR                   (GstShmCapture, --source shm)
```

- Container runs with **`ipc: host`** (POSIX shm lives in `/dev/shm`).
- The container's `cv2` has **no GStreamer backend**, so frames are pulled via **PyGObject (Gst appsink)** and wrapped in a `cv2`-compatible reader — the main loop is unchanged.
- The container needs **no Argus/EGL/nvidia camera libs**; shm buffer ≈ 20 MB.

Approach	Container runs AI	Reproducibility	Verdict
A: camera into container	—	EGLDisplay wall	❌
B: pipeline on host	❌	needs full host deps	works, not containerized
C: shm bridge (shipped)	✅	`compose up` + 1 host command	✅

Reproduce: `./deploy/start_camera_bridge.sh &` then `cd deploy && IMAGE_TAG=<tag> docker compose up`.